

The Latency Physics of Geo-Distributed Autoregressive Decode

Hanzo AI — ML Systems

An analytical systems paper — companion to the cross-backend inference measurements

Abstract

Splitting a large language model across machines that sit far apart is often imagined as “BitTorrent for AI”: pool everyone’s memory and the model runs. For *loading* weights and for *batched throughput* that intuition is right. For *single-stream, interactive decode* it is exactly wrong, and this paper derives why. We separate the two costs of a node boundary. **Bandwidth is trivial**: the only tensor that crosses a pipeline boundary is the hidden state for the in-flight tokens, $d_{\text{model}} \times n_{\text{tok}} \times b$ bytes — ~ 32 KiB for one token of a 700B-class model — which serializes in ≈ 0.1 ms over our measured 280 MB/s wired mesh, and the KV cache never crosses at all. **Latency is the wall**: autoregressive decode is a strict dependency chain, so each generated token must traverse every node once, and the next token cannot begin until the current one returns. Per-token decode rate is therefore $r = 1/(C + T_{\text{ring}})$, where C is the summed compute and $T_{\text{ring}} = \sum_i \ell_i$ is the one-way latency of a full loop around the ring. Because T_{ring} adds to a per-token denominator, a geo-distributed ring (Hong Kong $\rightarrow$ Dublin $\rightarrow$ New York, $T_{\text{ring}} \approx 280$ ms) floors single-stream decode at ≈ 3 tok/s *regardless of GPU speed* — the wall is the speed of light in fiber, not the accelerator. The headline result is the cure: **speculative decoding converts K sequential network traversals into one**. A local draft model proposes K tokens at full LAN speed; the distributed model verifies all K in a single pipeline pass; at acceptance rate α the effective rate is $r_{\text{spec}} = \frac{1-\alpha^{K+1}}{1-\alpha} / (Kc_d + C + T_{\text{ring}})$, which recovers 12–21 tok/s on the same 280 ms ring — interactive speeds from an intercontinental deployment. We close with four placement laws, including a derivation that pipeline parallelism ($O(\text{nodes})$ hops/token) strictly dominates tensor parallelism ($O(\text{layers})$ all-reduces/token) over any slow link by a factor of $\sim 4L/N$. Every model here is analytical and labelled as such; the one measured number is the 280 MB/s link.

1 Two regimes: dependency-bound vs. throughput-bound

The whole paper turns on one distinction, so we state it first.

A computation is **throughput-bound** when its pieces are independent: they can be done in any order, in parallel, and the only thing that matters is aggregate bytes-or-flops per second. Network *latency* barely touches such a workload — a slow link means each piece arrives a little later, but no piece waits on another, so the pipe stays full and throughput is set by *bandwidth*. Downloading a file is the canonical case; so is a batch of a thousand independent inference requests.

A computation is **dependency-bound** when its pieces form a chain: piece $t + 1$ cannot start until piece t finishes. Now latency is on the critical path of *every* step and cannot be hidden, because there is no independent work to overlap it with. Bandwidth is almost irrelevant — the messages are tiny — and the wall is the round-trip time.

Autoregressive decode is the purest dependency chain in production machine learning. Token $t + 1$ is sampled from the model’s output after token t has been fed back as input; *within* a token,

layer $\ell + 1$ consumes the output of layer ℓ . There is exactly one token in flight for a single stream, so there is nothing to overlap. Distributing that chain across nodes does not add parallelism to a single stream — it adds hops to the chain. This is why the “pool everyone’s GPUs” picture, which is a throughput picture, gives the wrong answer for interactive decode, which is a dependency problem. The rest of the paper makes both halves quantitative.

Parameters and provenance. Table 1 lists the symbols and the reference values used in the worked numbers. *One* value is measured — the 280 MB/s link goodput, from iperf3 on our direct 2.5 GbE mesh. The representative one-way fiber latencies are physical (Section 3) and are not our measurements; the model geometry (d_{model} , L) is that of a public 700B-class dense/MoE model; the local decode ceiling r_0 and draft rate r_d are stated parameters, not claims about a specific box. Every rate, table, and plot downstream is a closed-form evaluation of the models in this paper, not an experiment.

Table 1: Symbols and reference values. Only W is measured.

Symbol	Meaning	Reference value	Provenance
N	pipeline nodes (stages)	3	scenario
L	transformer layers	~ 100	700B-class model
d_{model}	hidden size	16384	700B-class model
b	activation bytes/element (bf16)	2	format
W	per-link goodput	280 MB/s	measured (2.5 GbE)
ℓ_i	one-way latency of hop i	40–130 ms (geo)	physical (fiber)
T_{ring}	ring latency $\sum_{i=1}^N \ell_i$	280 ms (geo)	physical (fiber)
$C = 1/r_0$	target compute per decode step	33 ms ($r_0=30$)	parameter
$c_d = 1/r_d$	local draft time per token	5 ms ($r_d=200$)	parameter
α	speculative acceptance rate	0.6–0.9	parameter
K	draft length (tokens/round)	1–24	swept

2 The crossing tensor: bandwidth is trivial

In pipeline parallelism (PP) the model is partitioned by *layer*: node i owns a contiguous stack of L/N layers. The only data that crosses the boundary between node i and node $i+1$ is the residual-stream hidden state at that layer cut — one activation tensor of shape $[n_{\text{tok}}, d_{\text{model}}]$ in the activation dtype. Its size is

$$B_{\text{hop}} = n_{\text{tok}} \cdot d_{\text{model}} \cdot b \text{ bytes}, \quad (1)$$

and the time to push it across a link of goodput W is $t_{\text{xmit}} = B_{\text{hop}}/W$. Three properties of Eq. (1) decide the paper.

It is independent of model depth. B_{hop} contains no L . Only the *width* d_{model} and the number of in-flight tokens appear, because exactly one activation crosses each cut no matter how many layers sit behind it. A deeper model costs more *compute* per node but not more *bytes* per hop. This is the structural reason PP is the right WAN topology (Section 8).

The KV cache never crosses the wire. Each node holds the keys and values for *its own* layers and reuses them locally on every step. The KV cache is the large, context- growing state; it stays put. What crosses is only the small, fixed-size residual vector. Distributing the model does not distribute the KV traffic — there is none across the boundary.

For decode it is a rounding error. For a single decoded token, $n_{\text{tok}} = 1$. With the 700B-class width $d_{\text{model}} = 16384$ and bf16 ($b = 2$),

$$B_{\text{hop}} = 1 \cdot 16384 \cdot 2 = 32,768 \text{ bytes} = 32 \text{ KiB}, \quad t_{\text{xmit}} = \frac{32,768}{280 \times 10^6} \approx 0.117 \text{ ms}$$

over our *measured* 280 MB/s mesh. Even at a serving-scale aggregate of 1000 tok/s each hop carries ~ 32 MB/s — about a tenth of a single 2.5 GbE link, and nothing on a datacenter WAN. **Bandwidth is never the constraint for decode.** (It can matter for large-prompt prefill, where $n_{\text{tok}} = P$: a 2000-token prompt crosses 65.5 MB per hop, ≈ 234 ms to serialize over 280 MB/s — but that is a bulk, streamable, latency-insensitive transfer that a fatter pipe fixes. It is a throughput cost, not a wall; Section 5.)

3 The wall: latency floors single-stream decode

Now the cost that does not go away. Generating one token drives the hidden state through all N nodes in sequence: node 1 embeds and runs its layers, hands off to node 2, and so on to node N , which produces the final hidden state; the model head samples the next token, which must return to node 1 to begin the next step. Counting hops on the critical path: $N - 1$ forward hand-offs plus one *return* hop that closes the loop, i.e. N inter-node latencies per token. Writing ℓ_i for the one-way latency of hop i and c_i for node i 's compute on its layer slice,

$$T_{\text{tok}} = \underbrace{\sum_{i=1}^N c_i}_C + \underbrace{\sum_{i=1}^N \ell_i}_{T_{\text{ring}}}, \quad r = \frac{1}{T_{\text{tok}}} = \frac{1}{C + T_{\text{ring}}}. \quad (2)$$

Two facts about Eq. (2) carry the argument.

PP gives a single stream zero compute speedup — only added latency. For one in-flight token, only one node computes at a time; the other $N - 1$ idle (the pipeline is empty). So the total compute $C = \sum_i c_i$ equals the monolithic single-node decode compute; splitting the model does not make one stream's arithmetic faster. What splitting *adds* is T_{ring} . PP buys memory capacity (a 700B model that fits nowhere else) and, with batching, throughput (Section 5) — never single-stream latency.

Co-locate the turn so the ring closes in exactly N hops. Keep the embedding, the first layers, the LM head, and the sampler on the same entry node. Then the “sample $\rightarrow$ embed next token” turnaround happens locally, with no network step, and the return is a single hop. Split the head from the embedding and you pay an extra hop every token (hidden state to the sampler node, then the token id back to the embedder). One node owns both ends of the ring.

The geo table. Fix a fast local ceiling $r_0 = 30$ tok/s ($C = 33$ ms) and sweep the ring latency. Table 2 and Figure 1 evaluate Eq. (2).

Table 2: Single-stream decode vs. ring latency, from $r = 1/(C + T_{\text{ring}})$ with $C = 33$ ms ($r_0 = 30$ tok/s). Analytical.

Deployment (3-node ring)	T_{ring} (ms)	decode (tok/s)	% of r_0
Single node (no network)	0	30.0	100%
Rack LAN (3×0.1 ms)	0.3	29.7	99%
Same metro (3×0.7 ms)	2	28.3	94%
Regional, one continent (3×7 ms)	20	18.8	63%
Continental US (3×20 ms)	60	10.7	36%
Two-continent (3×50 ms)	150	5.5	18%
Global ring HK→Dublin→NY	280	3.2	11%

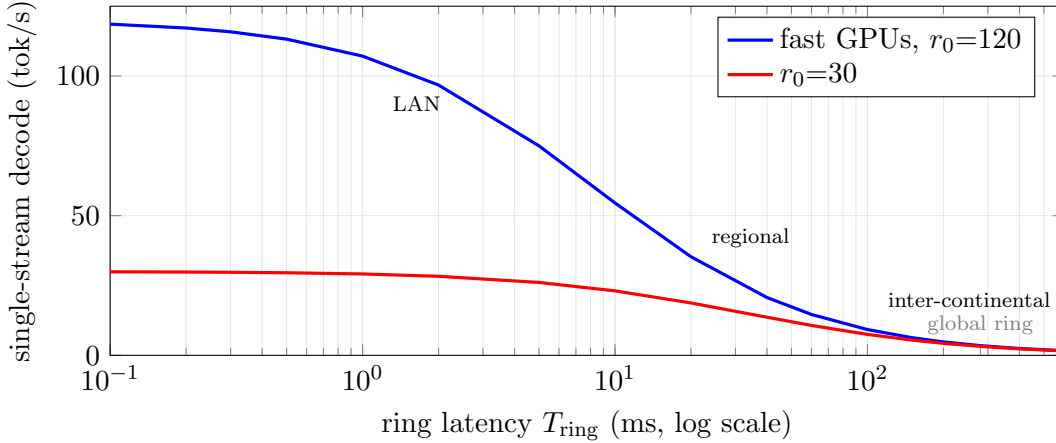


Figure 1: The wall. Two accelerator ceilings, $r_0 = 30$ and $r_0 = 120$ tok/s, collapse onto the *same* curve as T_{ring} grows: past ~ 100 ms the two are within ~ 2 tok/s. Beyond a few tens of milliseconds of ring latency, single-stream decode is set by the network, not the GPU. Analytical ($r = 1/(C + T_{\text{ring}})$).

The GPU stops mattering. The most important reading of Figure 1 is the convergence of the two curves. Hold $T_{\text{ring}} = 280$ ms and vary the local ceiling: $r_0 = 15 \rightarrow 2.9$, $30 \rightarrow 3.2$, $60 \rightarrow 3.4$, $120 \rightarrow 3.5$, $1000 \rightarrow 3.6$ tok/s. A $67\times$ faster accelerator buys a $1.2\times$ faster stream. Once the ring is long, $r \rightarrow 1/T_{\text{ring}}$ and the accelerator is a rounding error. Buying H100s to serve one interactive user over a global ring is spending on the wrong axis.

Why it is *physics*. T_{ring} has a hard floor: light in fiber travels at $c/n \approx 2.04 \times 10^5$ km/s ($n \approx 1.47$), i.e. $\approx 4.9 \mu\text{s}/\text{km}$ one-way, so a real (non-geodesic) route accrues $\gtrsim 5\text{--}7$ ms per 1000 km. Hong Kong to New York is $\sim 13,000$ km great-circle; no switch, protocol, or budget reduces that hop below ~ 110 ms one-way. The 130/40/110 ms legs of our example ring are representative real fiber latencies, and they are irreducible. This is the sense in which the title is literal: the wall is a property of space and light, and the systems question is not how to break it but how to *stop paying it per token* — which is Section 6.

4 “BitTorrent for AI” is half-right

The appeal of swarm inference — Petals-style pooling of volunteer GPUs across the internet — rests on an analogy to BitTorrent [3]. The analogy is exact where it applies and misleading where it does not, and the dividing line is precisely Section 1’s.

BitTorrent is fast because a file’s chunks are *independent*. A peer fetches them in any order, from many sources at once; a high-latency peer just means a given chunk lands a little later, never that another chunk waits. File download is throughput-bound and latency-insensitive — the ideal case for pooling far-flung capacity. **Everything about serving a distributed model that is throughput-bound inherits this good behaviour:** shipping the weights to the nodes, prefilling a long prompt, and running a large batch of independent requests (Section 5). For these, a geographically spread swarm is genuinely attractive.

Single-stream decode is the opposite object. Its “chunks” — the per-token forward passes — are *maximally dependent*: each is the input to the next. There is no order but the one order, no parallel sources, nothing to overlap the latency against. Pooling distant GPUs does not add throughput to one stream; it lengthens the dependency chain by the distance between the pools. The BitTorrent intuition, applied here, predicts the exact opposite of what happens. So the slogan is half-right: **BitTorrent for AI works for weight distribution, prefill, and batched throughput, and breaks for interactive single-stream latency** — and it breaks hardest precisely where users feel it, in the token-by-token cadence of a chat.

5 The regime map: where geo-distribution wins

The same 280 ms ring is cheap or catastrophic depending on how many tokens amortize it.

Prefill / TTFT amortizes the traversal over the whole prompt. A prompt of P tokens goes through the pipeline in *one* pass. The ring latency is paid once, not per token:

$$T_{\text{TTFT}} \approx C_{\text{prefill}}(P) + T_{\text{ring}}. \quad (3)$$

For a 2000-token prompt whose prefill compute is $\sim 1\text{--}2$ s on a big model, a +280 ms ring is a 15–30% tax on time-to-first-token — noticeable, not fatal. Per prompt-token the traversal is $280\text{ ms}/2000 \approx 0.14\text{ ms}$: amortized to nothing. Prefill is throughput-bound (Section 4); a geo ring is fine for it.

Batched throughput fills the pipe. With continuous batching, N pipeline stages run on N different requests at once. Each *individual* stream still crawls at Eq. (2)’s $r = 1/(C + T_{\text{ring}})$, but the *aggregate* scales with pipeline depth toward $\approx N \cdot (\text{per-stage rate})$ until a stage saturates. This is the BitTorrent regime done right: many independent streams are the independent chunks that keep the pipe full. High aggregate tok/s, high per-stream latency — excellent for offline/batch, useless as a proxy for one user’s experience.

The map (Table 3) is the practical statement of the thesis: only the bottom row is hard, and it is hard because it is the only dependency-bound row. The next section attacks exactly that row.

6 The cure: speculative decoding amortizes the traversal

The wall is that decode pays T_{ring} *per token*. Speculative decoding [1, 2] changes the denominator: it makes the network pay T_{ring} *per round*, where a round produces several tokens. It is the one lever that attacks the dependency structure itself rather than the constants.

Table 3: The regime map. Geo-distribution is bandwidth-cheap everywhere; it is latency-cheap only where many tokens share one traversal.

Workload	Bound by	Geo ring 280 ms?	Why
Weight distribution	throughput	great	independent chunks (BitTorrent)
Prefill / TTFT	throughput	fine	one traversal per prompt (Eq. 3)
Batched serving (aggregate)	throughput	great	depth fills the pipe, $\sim N \times$
Single-stream decode	dependency	hard	one traversal <i>per token</i> (Eq. 2)

Mechanism. A small *draft* model runs *locally* on the entry node — no network in its loop — and autoregressively proposes K candidate tokens. Those K tokens are then fed as a length- K sequence to the large distributed *target* model, which verifies them in a *single* pipeline pass, prefill-style: the K candidates flow through the ring together, one traversal, not K . The target accepts the longest correct prefix under the standard speculative-sampling rule (which preserves the target’s exact output distribution) and corrects the first mismatch. The expensive thing — the trip around the ring — happens once per round instead of once per token.

Expected tokens per round. Let each draft token be accepted independently with probability α (the empirical acceptance rate). The number of accepted drafts A satisfies $\Pr[A \geq j] = \alpha^j$ for $j \leq K$, so $\mathbb{E}[A] = \sum_{j=1}^K \alpha^j = \alpha \frac{1-\alpha^K}{1-\alpha}$. The target always contributes one further token (the correction on the first reject, or a bonus token if all K are accepted), giving the closed form of Leviathan et al. [1]:

$$\Gamma(\alpha, K) = 1 + \mathbb{E}[A] = \frac{1 - \alpha^{K+1}}{1 - \alpha} \quad \text{expected tokens per round.} \quad (4)$$

Effective rate. A round costs one local draft of K tokens plus one target verification pass (one traversal of the ring):

$$T_{\text{round}} = \underbrace{K c_d}_{\text{local draft}} + \underbrace{C_{\text{verify}}(K) + T_{\text{ring}}}_{\text{one target traversal}}, \quad r_{\text{spec}}(\alpha, K) = \frac{\Gamma(\alpha, K)}{T_{\text{round}}}. \quad (5)$$

In the memory-bandwidth-bound decode regime, verifying $K \leq 16$ tokens streams the weights the *same* once as a single decode step, so $C_{\text{verify}}(K) \approx C$ — the reason speculation is nearly free on the compute side.¹ Substituting the closed form,

$$r_{\text{spec}}(\alpha, K) = \frac{1}{K c_d + C + T_{\text{ring}}} \cdot \frac{1 - \alpha^{K+1}}{1 - \alpha} \quad (6)$$

The structure is the whole point: T_{ring} sits in the denominator once, divided into by $\Gamma(\alpha, K)$ tokens. The effective per-token network cost falls from T_{ring} to $T_{\text{ring}}/\Gamma(\alpha, K)$.

Numbers on the global ring. Take the intercontinental $T_{\text{ring}} = 280$ ms, target $C = 33$ ms, and a fast local draft $c_d = 5$ ms ($r_d = 200$ tok/s). Baseline single-stream decode is $1/(0.033 + 0.280) = 3.2$ tok/s. Table 4 and Figure 2 evaluate Eq. (6).

¹For large K the verify pass eventually becomes compute-bound (prefill-like) and $C_{\text{verify}}(K)$ grows with K ; that is one of the two forces creating an optimal K below, the other being the geometric decay α^K of the marginal acceptance. We take $C_{\text{verify}} \approx C$ over the plotted range $K \leq 24$, which is accurate for a 700B-class model where 24 tokens is still far below the arithmetic-intensity knee.

Table 4: Speculative decode on the 280 ms global ring, from Eq. (6) ($C=33$ ms, $c_d=5$ ms). Γ = expected tokens/round; r = effective tok/s. Baseline (no speculation) = 3.2 tok/s. Analytical.

α	$K = 4$		$K = 8$		$K = 16$		best
	Γ	r	Γ	r	Γ	r	tok/s (K^*)
0.6	2.31	6.9	2.47	7.0	2.50	6.4	7.1 ($K=6$)
0.7	2.77	8.3	3.20	9.1	3.33	8.5	9.1 ($K=8$)
0.8	3.36	10.1	4.33	12.3	4.89	12.4	12.7 ($K=12$)
0.9	4.10	12.3	6.13	17.3	8.33	21.2	21.6 ($K=21$)

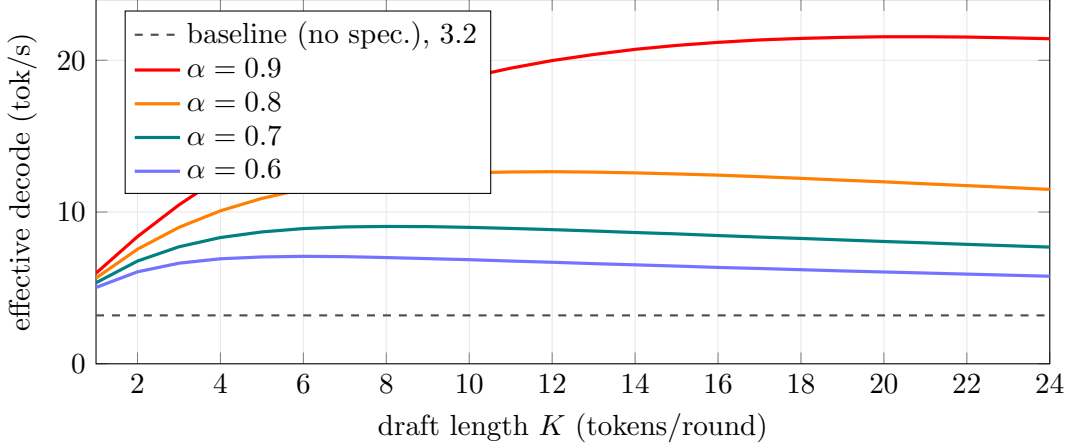


Figure 2: The cure. Effective single-stream decode on the *same* 280 ms global ring, from Eq. (6). Speculation lifts the 3.2 tok/s baseline (dashed) to 7–21 tok/s. Each curve has an optimum K : too small wastes the traversal, too large is drowned by the geometric decay α^K and the growing draft cost Kc_d . Higher acceptance α both raises the ceiling and pushes K^* out. Analytical.

Reading the result. On a ring that floors naive decode at 3.2 tok/s, a draft with acceptance $\alpha = 0.8$ recovers ~ 12.7 tok/s ($4.0\times$) and $\alpha = 0.9$ recovers ~ 21.6 tok/s ($6.8\times$) — from a bad slideshow to a readable stream, on the identical intercontinental deployment, with no faster hardware and no shorter cable. The mechanism is not a constant-factor tuning; it is a change in what the network is charged for. Naive decode pays one traversal per emitted token; speculation pays one traversal per $\Gamma(\alpha, K)$ emitted tokens, and Γ is exactly the knob acceptance gives you. This is why speculative decoding is *the* lever for interactive geo-distributed serving, not one option among several: it is the only cure that divides the wall instead of chipping at it.

The optimum is real and worth hitting. Each curve in Figure 2 peaks. Below K^* the fixed T_{ring} is shared over too few tokens; above it, marginal drafts are accepted with probability $\alpha^K \rightarrow 0$ while still costing draft time c_d and, eventually, verify compute. The peak drifts right with α ($K^* : 6 \rightarrow 8 \rightarrow 12 \rightarrow 21$ for $\alpha = 0.6 \rightarrow 0.9$) because a better draft keeps paying off for longer. A deployment should tune K to its measured acceptance and ring, not to a fixed default — the difference between $K=4$ and K^* is $\sim 1.7\times$ at $\alpha = 0.9$.

7 Placement laws

Four heuristics fall directly out of the models above; the first two are strategy, the last two are free.

1. **Regional-first placement.** T_{ring} dominates r (Eq. 2), so pack the whole pipeline into one region before spanning regions. A same-metro ring ($T_{\text{ring}} \approx 2\text{ ms}$) is 94% of local; a global ring is 11%. Span a WAN only when memory capacity forces it, and when you do, expect to pay for speculation (Section 6) to stay interactive. LAN neighbours are near-native; distant neighbours are a different regime.
2. **Speculate across every slow boundary.** Any ring above a few tens of milliseconds should run a local draft. It is the only lever that touches the T_{ring} -per-token term, and it composes with placement: regional-first shortens T_{ring} , speculation amortizes whatever remains.
3. **Co-locate embed + first layers + head + sampler on the entry node** (free). Makes the sample→embed turnaround local and closes the ring in exactly N hops instead of $N+1$ (Section 3). One node owns both ends.
4. **Keep the KV cache local to each node’s layers** (free, and automatic in PP). The KV cache never crosses the wire; only the 32 KiB hidden state does (Section 2). Do nothing that would ship KV state between nodes per step — it is the large state, and it has no reason to move.

8 Why pipeline-, not tensor-, parallel over slow links

The choice of *how* to split the model is not free, and over a WAN it is decisive. The two options generate fundamentally different network traffic per token.

Pipeline parallel (PP) splits by layer. Per token, the hidden state crosses each node boundary once: N point-to-point hops on the critical path, each moving the 32 KiB of Eq. (1). Critical-path network latency per token = $T_{\text{ring}} = \sum_{i=1}^N \ell_i$.

Tensor parallel (TP) splits each layer’s matrices across nodes [5]. Every layer must *all-reduce* partial results across all N nodes to reconstruct the activation — roughly two collectives per layer (one after attention, one after the MLP). That is $\sim 2L$ global synchronizations per token, each of which, even counted optimistically as a single round-trip ($\geq 2\bar{\ell}$; a ring all-reduce is worse, $2(N-1)$ serial steps), puts a barrier on the critical path. Critical-path network latency per token $\gtrsim 2L \cdot 2\bar{\ell} = 4L\bar{\ell}$.

The traffic law. Comparing the two, with $\bar{\ell}$ the mean one-way hop and $T_{\text{ring}} \approx N\bar{\ell}$:

$$\frac{(\text{TP latency/token})}{(\text{PP latency/token})} \gtrsim \frac{4L\bar{\ell}}{N\bar{\ell}} = \frac{4L}{N}. \quad (7)$$

PP costs $O(\text{nodes})$ hops per token; TP costs $O(\text{layers})$ collectives per token. Since a model always has far more layers than nodes ($L \gg N$) and each TP collective is a global barrier rather than a single hand-off, PP wins by a wide, structural margin: for $L = 100$, $N = 3$, Eq. (7) gives $\gtrsim 130\times$ less critical-path latency, and it is a lower bound (a realistic ring all-reduce adds another $\sim (N-1)$ factor). TP also moves more bytes ($\sim 2d_{\text{model}}b$ per node per collective, $\times 2L$), but latency alone settles it. TP belongs *inside* a node or an NVLink domain, where $\bar{\ell}$ is nanoseconds; across any link where $\bar{\ell}$ is milliseconds, TP multiplies the wall by $\sim 4L/N$ and PP is the only sane topology. This is the structural companion to Section 2: PP’s crossing tensor is

depth-independent precisely because it hands off once per node, and that is the same property that makes it latency-cheap.

9 What this paper claims, and what it does not

This is an analysis paper, and the analytical models *are* the contribution — so the line between derived and measured is drawn explicitly.

Measured (one number). The 280 MB/s link goodput is real: iperf3 over a direct 2.5 GbE cable on our mesh, sustained TCP (≈ 2.24 Gbit/s). It anchors the bandwidth claim in Section 2 and nothing else.

Physical (not our measurement). The one-way fiber latencies (40–130 ms legs, 280 ms ring) are representative of real intercontinental routes and are bounded below by the speed of light in fiber ($\approx 4.9 \mu\text{s}/\text{km}$). We cite the floor, not a traceroute we ran.

Parameters (stated, not claimed). $r_0 = 30$ tok/s, $r_d = 200$ tok/s, $\alpha \in [0.6, 0.9]$, $d_{\text{model}} = 16384$, $L \approx 100$. These are reference values chosen to be representative of a 700B-class model with a small local draft; every table and plot is a closed-form evaluation at these values, and the *shape* of every result (the $1/(C + T_{\text{ring}})$ wall, the T_{ring}/Γ amortization, the $4L/N$ law) is independent of the specific numbers.

Not claimed. We report no end-to-end wall-clock from a running geo-distributed deployment; no measured acceptance rates for a specific draft/target pair; no serving-throughput sweep; no overlap/async-speculation optimizations (which only improve the cure). Real acceptance depends on the draft, the target, and the prompt distribution, and real rings have jitter and packet loss the constant- ℓ model omits. The models predict *regimes and ratios*, and those are what the paper stands behind.

10 Conclusion

Distributing a large language model across machines is bandwidth-cheap and latency-expensive, and the two costs live in different regimes. Bandwidth is trivial because the only tensor crossing a pipeline boundary is a ~ 32 KiB hidden state — depth-independent, KV-cache-free, ~ 0.1 ms on a commodity link. Latency is a wall because autoregressive decode is a strict dependency chain: every token traverses every node, the next cannot start until this one returns, and so single-stream decode is floored at $1/(C + T_{\text{ring}})$ — ≈ 3 tok/s on a global ring, *independent of the accelerator*, because the wall is the speed of light in fiber. The “BitTorrent for AI” picture is right for the throughput-bound work (weights, prefill, batch) and wrong for the one dependency-bound workload (interactive decode) — and that is the only workload that is hard. The cure is to change what the network is charged for: speculative decoding turns K sequential traversals into one, dividing the wall by the accepted-tokens-per-round $\Gamma(\alpha, K)$ and recovering 12–21 tok/s on the identical 280 ms ring. Combined with regional-first placement, a co-located ring turn, node-local KV, and pipeline- (never tensor-) parallelism over the slow links, geo-distributed inference is not a contradiction — it is a latency-engineering problem with a known solution. The physics sets the floor; speculation decides how often you pay it.

References

- [1] Y. Leviathan, M. Kalman, Y. Matias. *Fast Inference from Transformers via Speculative Decoding*. ICML 2023.

- [2] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, J. Jumper. *Accelerating Large Language Model Decoding with Speculative Sampling*. arXiv:2302.01318, 2023.
- [3] A. Borzunov, D. Baranchuk, T. Dettmers, M. Ryabinin, et al. *Petals: Collaborative Inference and Fine-tuning of Large Models*. ACL 2023 (System Demonstrations).
- [4] Y. Huang, Y. Cheng, A. Bapna, et al. *GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism*. NeurIPS 2019.
- [5] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, B. Catanzaro. *Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism*. arXiv:1909.08053, 2019.
- [6] D. Narayanan, A. Harlap, A. Phanishayee, et al. *PipeDream: Generalized Pipeline Parallelism for DNN Training*. SOSP 2019.
- [7] Hanzo AI. *Edge LLM Inference Across Commodity Accelerators: hanzo-engine at llama.cpp Decode Parity on Every Backend*. hanzo-engine white paper (companion, measured).